

CareSync

A Smart Doctor–Patient Management Application

Capstone Project Report

Android Application Development

Submitted by

M Jomi Maria Sijo

Trainee/Intern

G1230

1. Introduction

CareSync is an Android-based doctor–patient management application developed as a capstone project. It is designed to help doctors efficiently manage their patients, schedule and track appointments, monitor analytics, and stay updated with health tips — all from a single mobile interface.

The application is built using Kotlin and follows the MVVM (Model–View–ViewModel) architecture pattern, with Room as the local database, LiveData for reactive UI updates, and WorkManager for reliable background notification scheduling. The app targets doctors in small to mid-sized clinical settings who need a lightweight, offline-capable tool for day-to-day management.

Key Features of CareSync

- Login & Registration — secure token-based doctor authentication with unique CS-XXXX ID
- Patient Management — add, view, search, and filter patients by age group with voice input
- Patient ID & Visit History — unique PT-XXX patient codes with full visit timeline
- QR Code Patient ID — generate and share scannable patient QR cards
- Appointment Management — schedule appointments with live search and status filter
- Auto Visit from Appointment — completing an appointment auto-creates a visit record
- Notifications — automated reminders 1 hour before each appointment via WorkManager
- Analytics Dashboard — visual statistics with bar chart and completion rate
- Doctor Profile — view and edit professional details with photo
- Health Tips — live health article feed via Wikipedia API in a bottom sheet
- Dark Mode — full light/dark theme toggle with persistent preference
- Onboarding Screens — first-launch walkthrough using ViewPager2
- Pull to Refresh — swipe down to refresh patient list

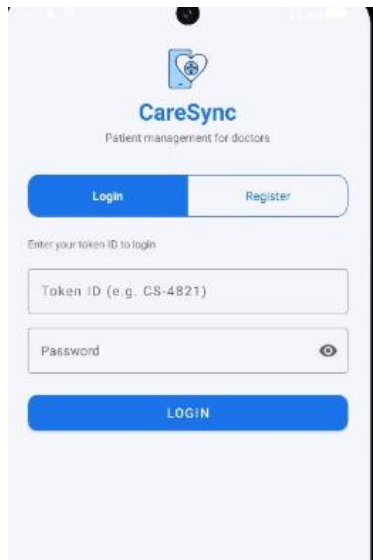
2. Technology Stack

Component	Technology
Language	Kotlin
Architecture	MVVM (Model–View–ViewModel)
UI	XML Layouts + Material Design Components
Database	Room
Reactive Data	LiveData, MediatorLiveData
Navigation	Android Navigation Component
Charts	MPAndroidChart v3.1.0
Networking	Retrofit + OkHttp
QR Code	ZXing (zxing-android-embedded 4.3.0)
Image Loading	Glide
Speech Recognition	Android SpeechRecognizer (built-in)
Auth Storage	SharedPreferences

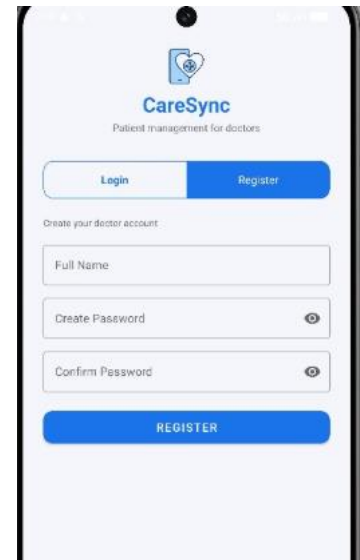
3. Features

3.1 Login & Registration

CareSync uses a token-based authentication system designed for doctors. On the Register tab, a new doctor enters their full name and sets a password. The app generates a unique CS token ID (e.g. CS-4821) which becomes their permanent login credential, stored locally in Room. On the Login tab, the doctor enters their CS token ID and password to authenticate. SharedPreferences stores the session so the app launches directly to the home screen on subsequent opens.



Login Screen



Register Screen

Token ID generation on registration:

```
// Generate unique CS token ID for new doctor

private suspend fun generateUniqueToken(): String {
    while (true) {
        val token = "CS-" + (1000..9999).random()
        if (db.doctorDao().checkTokenExists(token) == null) return token
    }
}
```

```

    }

}

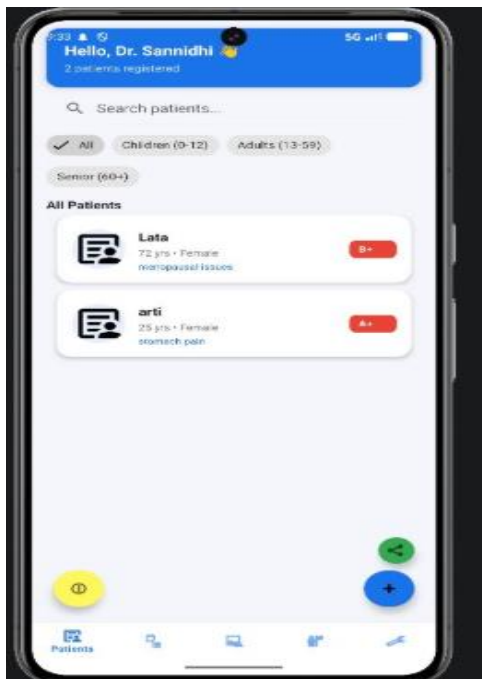
// Persist logged-in doctor session

prefs.edit().putInt("logged_in_id", doctor.id).apply()

```

3.2 Patient Management

The Patients screen is the home screen of CareSync. It displays all registered patients in a card-based list with name, age, gender, diagnosis, and blood group badge. Doctors can search patients by name, diagnosis, or Patient ID in real time and filter by age group using chips — Children (0–12), Adults (13–59), Senior (60+). The Add Patient form supports gallery photo pick, camera capture, and voice-to-text input for diagnosis and medication fields.



Patients List



Add New Patient

Patient age-group filter query (Room DAO):

```

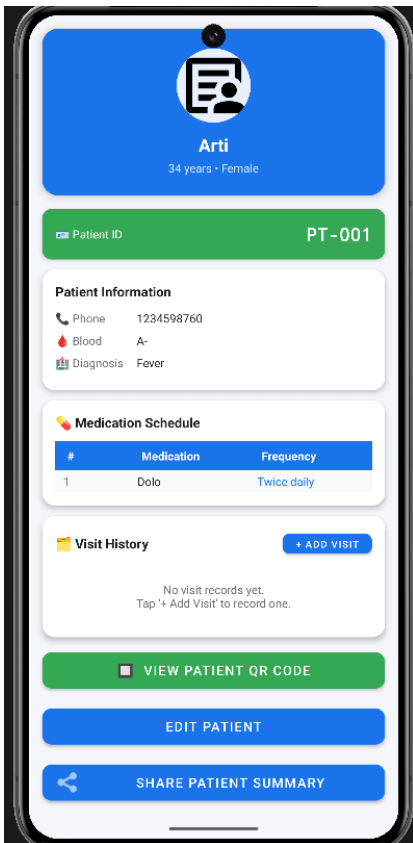
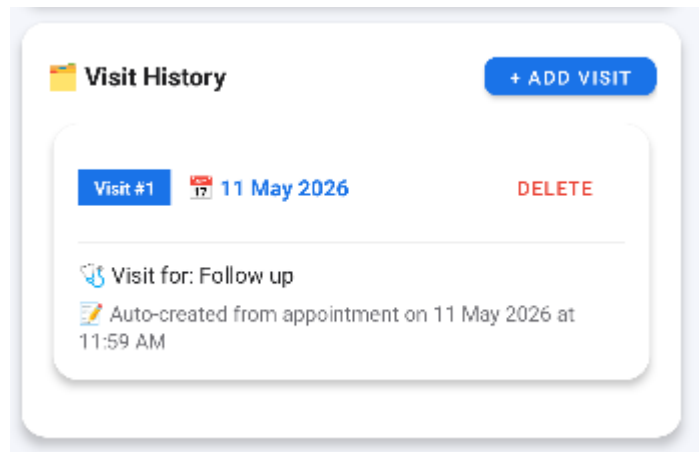
@Query("""
SELECT * FROM patients
WHERE (:query = " OR name LIKE '%' || :query || '%'
      OR diagnosis LIKE '%' || :query || '%'
      OR patientCode LIKE '%' || :query || '%')
AND (:ageGroup = " OR
      (:ageGroup = 'Children' AND age BETWEEN 0 AND 12) OR
      (:ageGroup = 'Adults'  AND age BETWEEN 13 AND 59) OR
      (:ageGroup = 'Senior'  AND age >= 60))
ORDER BY name ASC
""")
fun searchPatients(query: String, ageGroup: String): LiveData<List<Patient>>

```

3.3 Patient ID & Visit History

Every new patient is automatically assigned a unique Patient ID in PT-XXX format (e.g. PT-001, PT-002). This ID is generated by counting existing patients and verifying uniqueness against the database — handling edge cases where deletions could cause collisions. The Patient Detail screen prominently displays the ID on a colour-coded badge.

Each patient has a complete Visit History timeline. Visits are stored in a dedicated visits table linked to the patient via patientId. Visits are displayed in chronological order (oldest first) with a Visit #N badge, date, diagnosis, prescription, and notes. Doctors can add manual visit records or allow them to be created automatically when appointments are completed.

*Patient Record**Visit History***Unique Patient ID generation (collision-safe):**

```
private suspend fun generatePatientCode(): String {
    var code: String
    var count = db.patientDao().getPatientCount()
    do {
        count++
        code = "PT-%03d".format(count) // PT-001, PT-002...
    } while (db.patientDao().getPatientByCode(code) != null)
    return code
}
```

Visit entity (Room):

```

@Entity(tableName = "visits")
data class Visit(

    @PrimaryKey(autoGenerate = true) val id: Int = 0,

    val patientId: Int,    // links to Patient.id (one-to-many)

    val date: String,

    val diagnosis: String,

    val prescription: String,

    val notes: String,

    val doctorId: Int

)

```

DAO query — ordered ASC so oldest visit stays at top:

```

@Query("SELECT * FROM visits WHERE patientId = :patientId ORDER BY id ASC")
fun getVisitsForPatient(patientId: Int): LiveData<List<Visit>>

```

3.4 Auto-Visit from Completed Appointment

When a doctor marks an appointment as Completed, the app offers to automatically create a visit record — eliminating manual data entry. A confirmation dialog gives two choices: Complete & Add Visit (creates the record) or Just Complete (updates status only). The auto-created visit pre-fills the date from the appointment date and the diagnosis from the appointment reason, and includes a note indicating it was auto-generated. A snackbar with a View Patient shortcut appears immediately after.

Auto-creating visit when appointment is completed:

```

private fun markCompleteAndCreateVisit(appointment: Appointment) {

    lifecycleScope.launch {

        val visit = Visit(

```



```
        patientId = appointment.patientId,  
        date      = appointment.date,  
        diagnosis  = "Visit for: ${appointment.reason}",  
        prescription = "", // doctor fills later  
        notes     = "Auto-created from appointment at ${appointment.time}",  
        doctorId  = doctorId  
    )  
    db.visitDao().insertVisit(visit)  
    viewModel.update(appointment.copy(status = "Completed"))  
    NotificationScheduler.cancelReminder(requireContext(), appointment.id)  
}  
}
```

3.5 QR Code Patient ID

Each patient has a QR Code screen accessible from the Patient Detail page. The QR encodes all key patient data — ID, name, age, blood group, phone, and diagnosis — as plain text. Any phone camera can scan it, making it practical for sharing with reception staff or pharmacies. The ZXing library converts the text string to a BitMatrix (black/white pixel grid) and then to an Android Bitmap displayed in an ImageView.



Patient QR Code

Generating QR code with ZXing:

```
private fun generateQRCode(content: String) {  
    val bitMatrix = MultiFormatWriter().encode(  
        content,  
        BarcodeFormat.QR_CODE,  
        600, 600    // 600x600 pixel output  
    )  
    val bitmap = BarcodeEncoder().createBitmap(bitMatrix)  
    binding.ivQrCode.setImageBitmap(bitmap)  
}  
  
// Content encoded:  
// CareSync Patient | ID: PT-001 | Name: Arti  
// Age: 25 | Blood: A+ | Phone: 0987654321
```

3.6 Voice Input for Patient Notes

The Add Patient form has microphone FAB buttons next to the Diagnosis and Medication fields. Tapping either opens Android's built-in speech recognition dialog. The recognised text is automatically inserted into the relevant field — and if the field already has content, the new text is appended with a comma separator. This allows doctors to dictate medication lists naturally without typing.

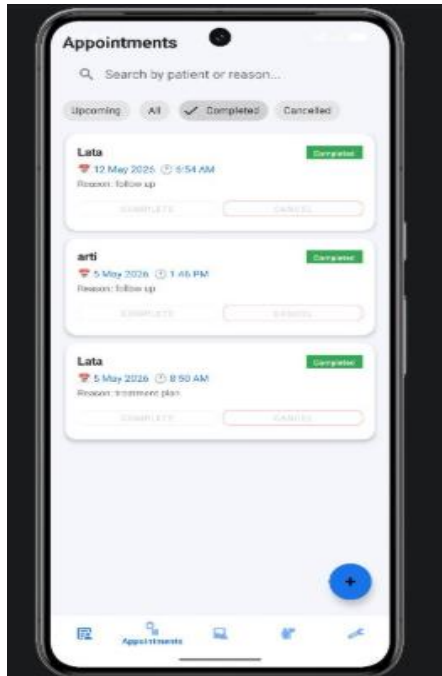
Launching speech recognition and handling result:

```
private fun startVoiceInput(prompt: String) {
    val intent = Intent(RecognizerIntent.ACTION_RECOGNIZE_SPEECH).apply {
        putExtra(RecognizerIntent.EXTRA_LANGUAGE_MODEL,
            RecognizerIntent.LANGUAGE_MODEL_FREE_FORM)
        putExtra(RecognizerIntent.EXTRA_PROMPT, prompt)
    }
    voiceLauncher.launch(intent)
}

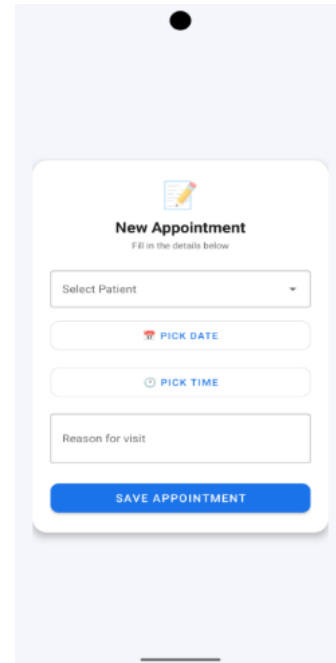
// Result handler — appends to existing text with comma
val spokenText = matches?.firstOrNull() ?: return
val existing = binding.etDiagnosis.text.toString().trim()
binding.etDiagnosis.setText(
    if (existing.isEmpty()) spokenText else "$existing, $spokenText"
)
```

3.7 Appointment Management — Search & Filter

The Appointments screen lists all appointments with patient name, date, time, reason, and a colour-coded status badge (blue = Upcoming, green = Completed, red = Cancelled). A live SearchView filters appointments in real time by patient name or reason. Four filter chips allow instant status filtering. Both search and chip filter work together simultaneously, powered by a Room SQL query with LIKE pattern matching and MediatorLiveData in the ViewModel.



Appointments Screen



Add New Appointment

Combined search + status filter (Room DAO):

```
@Query("""
SELECT * FROM appointments
WHERE (:query = " OR patientName LIKE '%' || :query || '%"
      OR reason LIKE '%' || :query || '%')
AND (:status = " OR status = :status)
ORDER BY date ASC, time ASC
""")
```

```
fun searchAppointments(query: String, status: String): LiveData<List<Appointment>>
```

MediatorLiveData — reacting to both search and chip filter:

```
val filteredAppointments = MediatorLiveData<List<Appointment>>().apply {
    var currentQuery = ""; var currentStatus = "Upcoming"
    var currentSource: LiveData<List<Appointment>>? = null
    fun refresh() {
        currentSource?.let { removeSource(it) }
        val src = repository.searchAppointments(currentQuery, currentStatus)
        currentSource = src; addSource(src) { value = it }
    }
    addSource(_searchQuery) { q -> currentQuery = q; refresh() }
    addSource(_statusFilter) { s -> currentStatus = s; refresh() }
}
```

3.8 Appointment Notifications

CareSync schedules a push notification exactly 1 hour before each upcoming appointment using `WorkManager`. When an appointment is saved, the app parses the date and time strings into epoch milliseconds, calculates the delay, and enqueues a `OneTimeWorkRequest` tagged with the appointment ID. `WorkManager` jobs survive app closure and device restarts. Deleting or cancelling an appointment immediately cancels the corresponding `WorkManager` job to prevent stale reminders.

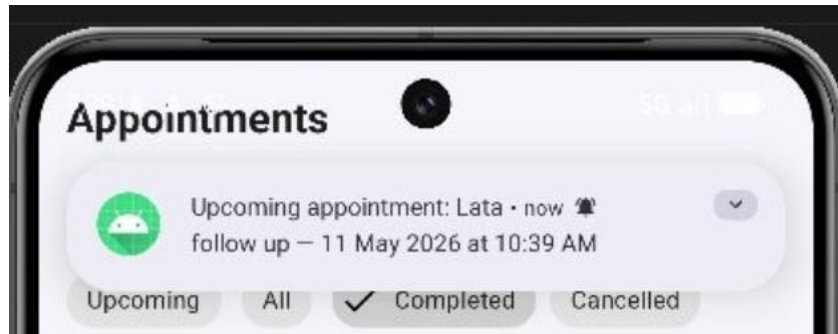


Figure: Push notification delivered 1 hour before appointment

Scheduling the reminder with WorkManager:

```
fun scheduleReminder(context: Context, appointment: Appointment) {
    val apptMs = parseAppointmentTime(appointment.date, appointment.time) ?: return
    val delay = apptMs - TimeUnit.HOURS.toMillis(1) - System.currentTimeMillis()
    if (delay <= 0) return

    val req = OneTimeWorkRequestBuilder<AppointmentReminderWorker>()
        .setInitialDelay(delay, TimeUnit.MILLISECONDS)
        .setInputData(workDataOf(
            "patientName" to appointment.patientName,
            "time"        to appointment.time
        ))
        .addTag("appt_${appointment.id}")
        .build()

    WorkManager.getInstance(context)
        .enqueueUniqueWork("appointment_${appointment.id}",
            ExistingWorkPolicy.REPLACE, req)
}
```

3.9 Analytics Dashboard

The Analytics screen provides a real-time overview of appointment statistics. Four stat cards display total, upcoming, completed, and cancelled counts. A horizontal progress bar shows the overall completion rate as a percentage. A bar chart (MPAndroidChart) visualises appointment frequency per date. A Most Frequent Patients section ranks the top 5 patients by visit count. All data updates automatically as appointments change.

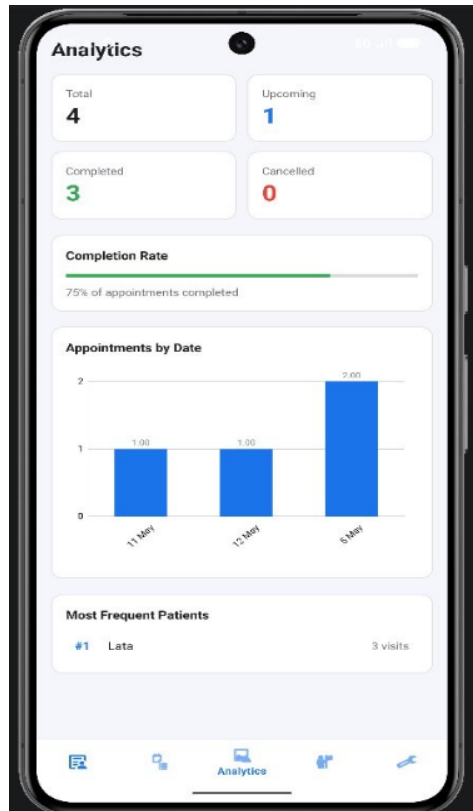


Figure: Analytics Dashboard

SQL aggregation queries for analytics (Room DAO):

```
// Count by status
```

```
@Query("SELECT COUNT(*) FROM appointments WHERE status = 'Completed'")
```

```
fun getCompletedCount(): LiveData<Int>
```

```
// Appointments grouped by date — feeds bar chart
```

```
@Query("SELECT date, COUNT(*) as count FROM appointments GROUP BY date  
ORDER BY date ASC")
```

```
fun getAppointmentsPerDay(): LiveData<List<DateCount>>
```

```
// Top 5 most frequent patients
```

```
@Query("SELECT patientName, COUNT(*) as count FROM appointments  
GROUP BY patientName ORDER BY count DESC LIMIT 5")
```

```
fun getTopPatients(): LiveData<List<PatientCount>>
```

3.10 Doctor Profile

The Profile screen displays the logged-in doctor's professional details: specialisation, department, experience, qualification, hospital, phone number, and bio. Data is fetched from Room using the stored login ID from SharedPreferences. An Edit Profile button opens the edit screen where all fields can be updated, and profile photos can be picked from gallery or taken with the camera. A Change Language button provides access to the language selection screen.

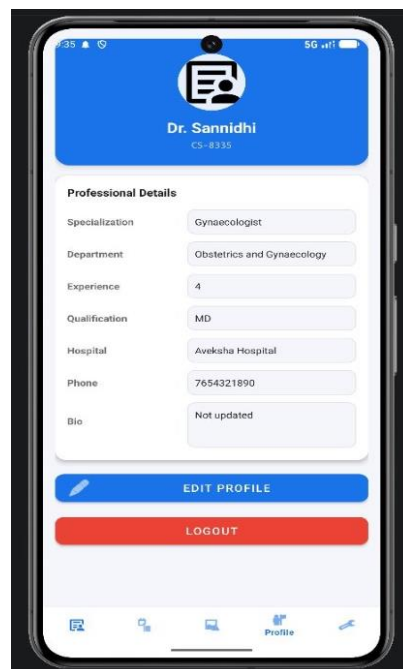


Figure: Doctor Profile Screen

3.11 Health Tips

The Health Tips feature presents a bottom sheet that slides up from the Patients screen via a floating action button. It displays a live feed of health tip articles fetched from the Wikipedia API using Retrofit. Each card shows a title, a brief excerpt, and a Read More button that opens the full article in an in-app WebView. A Refresh button reloads the feed with different health topics on demand.

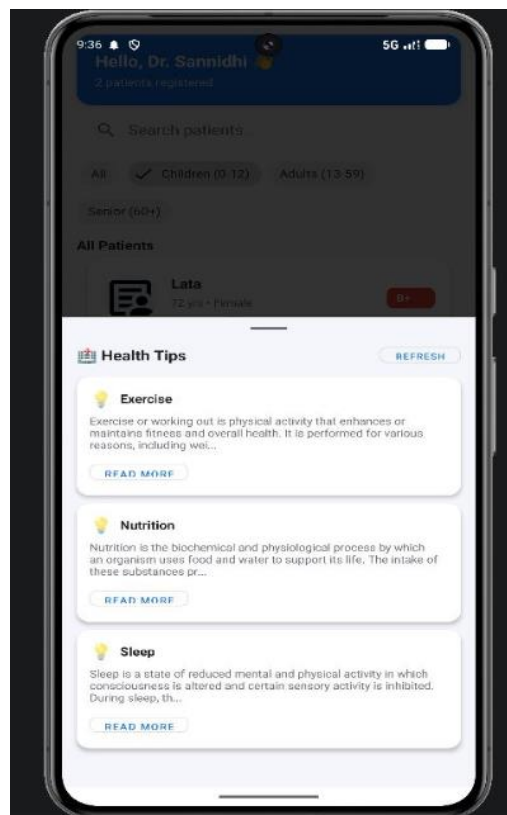


Figure: Health Tips Bottom Sheet

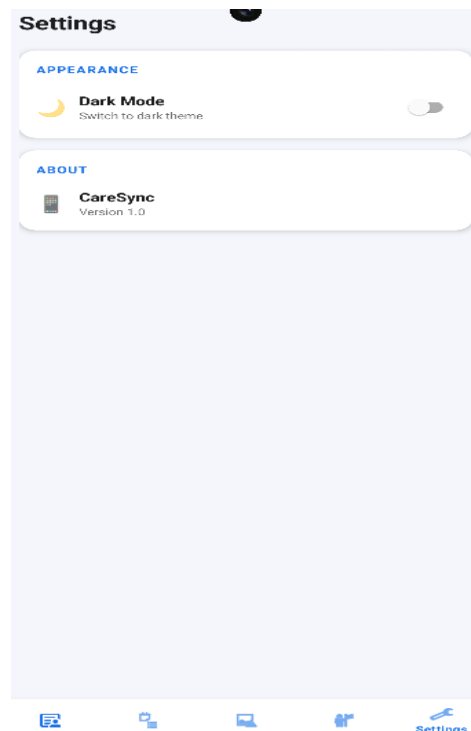
Retrofit call to Wikipedia API:

```
interface HealthApiService {
    @GET("api/rest_v1/page/summary/{topic}")
    suspend fun getHealthTip(@Path("topic") topic: String): HealthTipResponse
}
```

```
// Fetch multiple topics in parallel using coroutines  
  
val topics = listOf("Exercise", "Nutrition", "Sleep", "Mental_health")  
  
val tips = topics.map { healthApiService.getHealthTip(it) }
```

3.12 Dark Mode

The Settings screen includes a Dark Mode toggle that immediately switches the app theme using `AppCompatActivity.setDefaultNightMode()`. Android automatically applies the `values-night/` colour resources across all layouts. The user's preference is persisted in `SharedPreferences` so the theme is restored on every launch without any flash.



Dark Mode

Dark mode toggle with SharedPreferences persistence:

```
binding.switchDarkMode.setOnCheckedChangeListener { _, isChecked ->
```

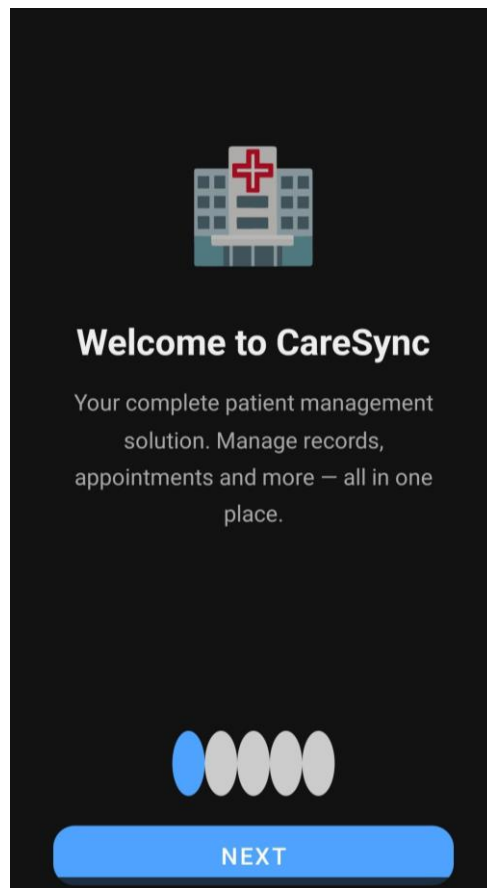
```
prefs.edit().putBoolean("dark_mode", isChecked).apply()

AppCompatActivity.setDefaultNightMode(
    if (isChecked) AppCompatActivity.MODE_NIGHT_YES
    else AppCompatActivity.MODE_NIGHT_NO
)
}

// res/values/colors.xml    → light mode colours
// res/values-night/colors.xml → dark mode colours (auto-applied by Android)
```

3.13 Onboarding Screens

A 5-slide onboarding walkthrough is shown only on first install using a SharedPreferences flag (onboarding_done). Each slide is a full-screen page in a ViewPager2 with title, subtitle, and illustration. A dot indicator (TabLayoutMediator) shows progress. The final slide's Next button becomes Get Started, which marks onboarding complete and launches the language selection screen.



Onboarding Screen 1

ViewPager2 + dot indicator setup:

```
binding.viewPager.adapter = OnboardingAdapter(pages)

TabLayoutMediator(binding.dotsIndicator, binding.viewPager) { _, _ -> }.attach()

binding.viewPager.registerOnPageChangeCallback(
    object : ViewPager2.OnPageChangeCallback() {
        override fun onPageSelected(position: Int) {
            binding.btnNext.text =
                if (position == pages.size - 1) "Get Started" else "Next"
```

```

    }

    }

)

// Mark done on final slide

prefs.edit().putBoolean("onboarding_done", true).apply()

```

3.14 Pull to Refresh

The Patients screen supports pull-to-refresh using `SwipeRefreshLayout`. Pulling down triggers a fresh Room LiveData observation. The loading spinner automatically stops when the observer fires with new data. This provides a familiar gesture-based refresh pattern without any extra buttons.

Pull to refresh implementation:

```

binding.swipeRefresh.setColorSchemeColors(
    getColor(R.color.primary), getColor(R.color.accent)
)

binding.swipeRefresh.setOnRefreshListener {
    switchObserver(patientViewModel.allPatients)
}

// Observer stops spinner automatically when data arrives

private val patientObserver = Observer<List<Patient>> { patients ->
    binding.swipeRefresh.isRefreshing = false
    adapter.submitList(patients)
}

```

4. Application Architecture

CareSync follows the MVVM (Model–View–ViewModel) architecture pattern recommended by Google for Android development. This separates concerns cleanly across four layers:

- UI Layer (Fragments & Activities) — observes LiveData from ViewModels and updates the interface reactively. Never accesses the database directly.
- ViewModel Layer — holds and processes UI-related data. Survives screen rotation. Exposes LiveData streams to the UI.
- Repository Layer — single source of truth. Abstracts Room DAO and Retrofit API calls from the ViewModel.
- Room Database Layer — local SQLite database with DAOs for Appointments, Patients, Doctors, and Visits.

Navigation between screens is handled by the Android Navigation Component with a bottom navigation bar wired via `setupWithNavController()`, which automatically manages the back stack and tab highlighting.

Layer	Responsibility
UI (Fragments/Activities)	Display data, handle clicks, show dialogs — never touches DB directly
ViewModel	Hold UI state, survive rotation, call repository, expose LiveData
Repository	Single source of truth — abstracts DB and API from ViewModel
Room DAO	SQL queries, returns LiveData for reactive updates
WorkManager	Background tasks independent of UI lifecycle
SharedPreferences	Lightweight persistent storage for session and settings

5. Android Concepts Used — Detailed

5.1 MVVM Architecture

MVVM is the standard Android architecture pattern recommended by Google. Every screen in CareSync follows MVVM. The Model is the Room DB (Patient, Appointment, Doctor, Visit entities). The ViewModel holds and manages data using LiveData. The View (Activities and Fragments) simply observes LiveData and updates the UI — it never touches the database directly.

The ViewModel survives screen rotation. If the doctor rotates their phone while viewing the patient list, the data does not reload — it is still in the ViewModel. Without this, every rotation would hit the database again causing unnecessary re-queries and visible loading states.

ViewModel and observer pattern:

```
class PatientViewModel(application: Application) : AndroidViewModel(application) {  
    val allPatients: LiveData<List<Patient>> = repository.allPatients  
  
    fun insert(patient: Patient) = viewModelScope.launch {  
        repository.insert(patient)  
    }  
}  
  
// Fragment observes — never touches DB directly  
viewModel.allPatients.observe(viewLifecycleOwner) { patients ->  
    adapter.submitList(patients)  
}
```

5.2 Room Database — Local Storage

Room is Google's recommended library for structured local data storage on Android, built on top of SQLite. It stores all patients, appointments, doctor accounts, and visit records. Room has three components: Entity (the table definition), DAO (the queries), and Database (the actual file).

Database migrations are used every time a new table or column is added, preserving all existing data. CareSync has gone through 7 migrations, adding columns for doctor profile, patient code, visit history, and more.

Database migration example — adding patientCode:

```
val MIGRATION_6_7 = object : Migration(6, 7) {
    override fun migrate(database: SupportSQLiteDatabase) {
        // Add patientCode to existing patients table
        database.execSQL(
            "ALTER TABLE patients ADD COLUMN patientCode TEXT NOT NULL
            DEFAULT ""
        )
        // Create visits table
        database.execSQL("""
            CREATE TABLE IF NOT EXISTS visits (
                id INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
                patientId INTEGER NOT NULL,
                date TEXT NOT NULL,
                diagnosis TEXT NOT NULL,
                prescription TEXT NOT NULL,
                notes TEXT NOT NULL,
                doctorId INTEGER NOT NULL
            ) """)
    }
}
```



```

        """)
    }
}

```

5.3 SharedPreferences

SharedPreferences is Android's built-in key-value storage for simple persistent data. CareSync uses it for three distinct purposes: storing the login session (`is_logged_in`, `logged_in_token`, `logged_in_id`), routing decisions in `SplashActivity` (`onboarding_done` flag, login state), and persisting user preferences (`dark_mode` toggle, `app_language` choice).

Without SharedPreferences, the doctor would have to log in every time they open the app. The session persists across app restarts and phone restarts until they explicitly log out.

5.4 Intents — Explicit and Implicit

Explicit intents navigate between specific screens (e.g. opening `PatientDetailActivity` with a `patient_id` extra). Implicit intents trigger system actions like sharing — Android shows a chooser with WhatsApp, Gmail, etc. The share feature allows doctors to send a patient summary to a specialist in one tap.

5.5 Runtime Permissions

Android requires apps to request permission before accessing camera, gallery, or microphone. CareSync handles three generations of Android's evolving permission model: `READ_EXTERNAL_STORAGE` for Android 12 and below, `READ_MEDIA_IMAGES` for Android 13+, and `RECORD_AUDIO` for voice input. The code checks the Android version (`Build.VERSION.SDK_INT`) and requests the correct permission for each device.

5.6 Fragments + Navigation Component

The four main tabs — Patients, Appointments, Profile, and Settings — are Fragments living inside `MainActivity`. The Navigation Component manages transitions via a `nav_graph.xml` that declares all destinations. A single line (`setupWithNavController`) connects the bottom navigation bar to the `NavController`, automatically managing back stack and tab highlighting.

ViewBinding in Fragments uses a nullable pattern (`_binding`) that is nulled in `onDestroyView` to prevent memory leaks — the Fragment can outlive its View, so keeping a reference to the View causes leaks.

5.7 RecyclerView + ListAdapter + DiffUtil

RecyclerView efficiently displays scrollable lists by recycling off-screen views. ListAdapter (a smarter RecyclerView.Adapter) uses DiffUtil to calculate exactly what changed between two lists. With 100 patients, when one name updates, only that one card redraws — not all 100. This is critical for smooth 60fps scrolling performance.

5.8 LiveData + Observer Pattern

LiveData is lifecycle-aware reactive data. Room DAOs return LiveData, which means when a new patient is inserted into the database, the LiveData automatically emits the updated list to every active observer — no manual refresh ever needed. Observers must use `viewLifecycleOwner` (not 'this') in Fragments, because the Fragment view can be destroyed while the Fragment itself still exists.

5.9 Coroutines

Kotlin coroutines allow asynchronous code without blocking the UI thread. Android throws `NetworkOnMainThreadException` if network or DB calls happen on the main thread. Every database operation in CareSync uses suspend functions called from `viewModelScope.launch` (tied to ViewModel lifecycle) or `lifecycleScope.launch` (tied to Activity/Fragment lifecycle). Both auto-cancel when their owner is destroyed, preventing memory leaks.

5.10 ViewBinding

ViewBinding generates a typed Kotlin class for every XML layout at compile time. This eliminates all `findViewById` calls and makes it impossible to reference a non-existent view ID — the build fails before the app can crash. In Fragments, the binding is nullable (`_binding`) and nulled in `onDestroyView` to prevent memory leaks.

5.11 Retrofit + OkHttp + Networking

Retrofit defines the Wikipedia API endpoint as a Kotlin interface. OkHttp handles the actual HTTP connection with a custom client that adds a User-Agent header (required by Wikipedia's API) and configures SSL. A sealed class (HealthTipState: Loading / Success / Error) cleanly manages the three possible API states, and the Fragment switches UI visibility based on which state is active.

5.12 WebView

WebViewActivity loads the full Wikipedia health article URL inside the app. WebViewClient keeps navigation inside the WebView (without it, tapping links opens Chrome). WebChromeClient monitors page load progress and hides the progress bar at 100%. The back button checks canGoBack() to navigate WebView history before closing the activity.

5.13 Glide — Image Loading

Glide loads patient and doctor profile photos from local URIs. It handles background thread loading, automatic disk and memory caching, safe resizing for any image size, and circleCrop() for the circular profile photo style. Using setImageURI directly would run on the main thread and could crash with large images — Glide solves all of these issues automatically.

5.14 ZXing — QR Code Generation

The ZXing library converts a text string into a QR code Bitmap entirely on-device with no internet required. MultiFormatWriter.encode() converts text to a BitMatrix (2D grid of black/white cells). BarcodeEncoder.createBitmap() converts the grid to a Bitmap. The result is displayed in a standard ImageView. Any QR scanner app can read the output — no special app is required on the patient's side.

5.15 SpeechRecognizer — Voice Input

Android's built-in RecognizerIntent.ACTION_RECOGNIZE_SPEECH opens Google's speech dialog. LANGUAGE_MODEL_FREE_FORM mode handles natural medical terminology like 'hypertension' or 'paracetamol'. The result is a ranked list of recognised matches — firstOrNull()

picks the highest-confidence result. A VoiceField enum tracks which text field (Diagnosis or Medication) triggered the mic so the result handler knows where to place the text.

5.16 WorkManager — Background Notifications

WorkManager is the library for guaranteed background work. Unlike Handler.postDelayed() which only works while the app is running, WorkManager persists through app closure, device restarts, and battery optimisation. Each appointment notification is tagged with appt_{id} so it can be individually cancelled when an appointment is deleted or cancelled. CoroutineWorker is used instead of basic Worker so suspend functions can be called without runBlocking.

5.17 Dark Mode

Android's DayNight theme system allows a single colour reference (e.g. @color/background) to automatically resolve to different values in light and dark mode. Light colours are defined in res/values/colors.xml and dark equivalents in res/values-night/colors.xml. When MODE_NIGHT_YES is set via AppCompatActivity, Android automatically switches all colour references across the entire app with zero layout changes required.

5.19 ViewPager2 — Onboarding

ViewPager2 is the modern replacement for ViewPager. It supports swipe gestures between pages and smooth animations. TabLayoutMediator creates the dot indicator that highlights as the user swipes. OnPageChangeCallback detects the last page so the button text changes from Next to Get Started. A SharedPreferences flag ensures onboarding is shown exactly once.

5.20 SwipeRefreshLayout — Pull to Refresh

SwipeRefreshLayout wraps the patient list and detects the downward pull gesture. The refresh spinner's colour is customised to match the app's primary and accent colours. The spinner stops automatically when the LiveData observer receives fresh data — no manual isRefreshing = false needed in most paths.

5.21 Token-Based Authentication

CareSync uses a custom authentication system where each doctor gets a unique CS-XXXX token instead of an email address. This is practical for small clinic settings where doctors may not have reliable email access. The token is generated by picking a random 4-digit number, verifying it is unique in the Room database (collision loop), and storing it as the doctor's permanent credential. Login verifies token + password against Room — no server or internet required.

5.22 FileProvider — Camera Integration

Android requires FileProvider to securely share files between apps. When launching the camera, a temp file is created in external pictures storage, and its URI is wrapped in FileProvider.getUriForFile() before being passed to the camera Intent via EXTRA_OUTPUT. Without FileProvider, modern Android versions would block the file access with a FileUriExposedException.

5.23 MediatorLiveData — Combined Filtering

MediatorLiveData can observe multiple LiveData sources and merge them into one output. The appointment filter needs to react to both the search query and the status chip — two independent inputs. Each time either changes, the old Room LiveData source is removed (removeSource) and a new one added with the updated parameters. Without removing the old source, multiple sources would stack up and cause duplicate or conflicting emissions.

5.24 TableLayout — Dynamic Rows

TableLayout organises content in rows and columns. The medication schedule in Patient Detail uses a TableLayout with a static header row (from XML) and dynamic data rows created in Kotlin. Each medication from the comma-separated string is split, and a TableRow with three TextViews (number, name, frequency) is added programmatically. Zebra striping (alternating row colours) is applied for readability.

6. Conclusion

CareSync successfully demonstrates a full-featured Android application built with modern Android components and best-practice architecture. The project covers the complete development lifecycle — from database design and XML UI layout through background notification processing, real-time analytics, QR code generation, voice input, and multi-language support — making it a comprehensive showcase of advanced Android development skills.

The application is fully functional with offline support via Room, reactive UI via LiveData, reliable background notifications via WorkManager, clean MVVM architecture, and an extensible codebase. The addition of patient visit history with auto-population from completed appointments, QR-based patient IDs, voice input, and three-language localisation elevate it well beyond a standard capstone project.

Future Enhancements

- Cloud synchronisation — backup patient data to Firebase or a REST API
- Multi-doctor support — allow multiple doctor accounts on one device
- PDF prescription generation — export prescriptions as printable PDFs
- AI-powered diagnosis assistant — integrate Claude API for symptom checking
- Biometric login — fingerprint authentication using BiometricPrompt API
- Patient-facing QR scanning — scan QR at reception to pull up patient record